Escape Barbershop — the whole thing, in one document

Everything below is one system: the shop's till and back office, a booking app for customers, a day screen for barbers, and a board for the wall. One database, one set of rules, one deployment.

This document is self-contained — it is the one to hand somebody who has not seen the project. The five others go deeper on a single question each; they are listed at the end.

Written 24 September 2026. The numbers are from the live shop that day.

---

1. What it is

A barbershop in Cairo already ran its counter, payroll and warehouse on this system. Booking, the customer app and the barber app were built on top of that, deliberately sharing its database rather than sitting beside it.

That decision is the reason for most of what follows. A booking that becomes a visit is the same customer, the same barber and the same money the shop was already counting — so the diary and the takings cannot drift apart, and nobody reconciles two systems at the end of the day.

Live, in the shop, today:

Customers on file	7,381
Visits recorded	6,832 (since February 2026)
Visits in the last 30 days	1,210
Active barbers	15
Services on the menu	52

---

2. What it does

The till and the back office (`/visits` , `/admin` , …)

The part the shop has used the longest, and the part that handles money.

- **Visits** — ring up a customer against a barber and a list of services; cash, card, InstaPay or any mix of the three, with tips tracked apart from revenue.
- **The cash drawer** — open, move money by hand, close against what was counted, and reconcile. No second session over an open one; nothing added to a shut drawer.
- **Expenses and advances**, charged to whichever account paid them — the drawer, the card, or the bank.
- **Payroll** — base salary, commission by percentage or per visit, penalties, advances and attendance, ending in a salary sheet.
- **The warehouse** — purchases, suppliers, issue notes to the shop floor, stocktakes. Nothing can be issued that is not there.
- **Reports** — the day journal, a period report, discounts given, absences, and an activity trail that outlives the thing it recorded and stays unreadable to the people it watches.
- **Insight screens** — retention, busiest hours and weekdays, new versus returning, barber utilisation.
- **Floor reports** — what happened during a cut, written by whoever was standing there, read by the owner alone.

The customer app (`/app`)

Signed in with a phone number. Dark by default, light on request, English and Arabic.

- **Book** — service, barber, time, confirm. Free times come from the same engine the shop's own diary uses.
- **The usual** — their habitual barber, cut and hour, read from their own history and offered back in one tap.
- **Is it worth walking down now** — the live queue when the shop is running one, otherwise what usually happens at this hour on this weekday.
- **Their haircuts** — every visit on file, with a receipt for each they can keep or send.
- **Reviews** — stars and a line, on any cut they actually had.
- **The cut they want** — photos the barber sees before a word is exchanged.
- **Price list and who's in** — both readable without an account at all.
- **Loyalty, referrals, birthdays** — a punch card, a code worth something to both sides, and a greeting once a year.
- **Reminders and confirmations** as phone notifications, which cost nothing to send.

The barber app (`/barber`)

Signed in with a phone number and a six-digit PIN, because barbers do not have work email addresses and a login nobody can perform is a feature nobody has.

- **Their day** — who is coming, who is in the chair, who never came.

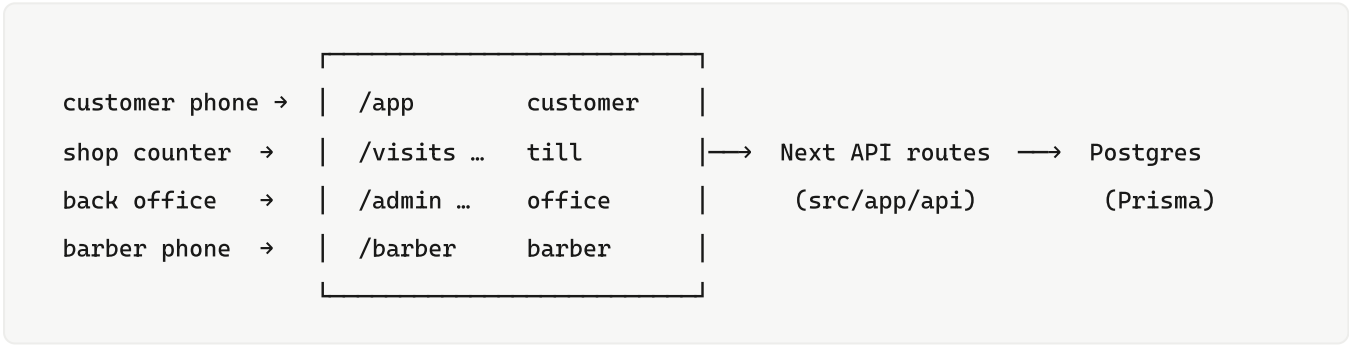
- **Their customers** — history, the usual, notes, and the photos that customer wants.
- **Their earnings** — their own commission and tips, never the shop's takings and never another barber's column.
- **Their portfolio** — up to twelve photos of their own work, shown to customers.
- **Their time off** — asked for here, approved in a tap by the owner. Asking blocks nothing until somebody answers.

The screen on the wall (`/wall`)

The live queue, the wait, and today's offers — written for the room rather than for a person holding a phone.

---

3. How it is built



Next.js 14 (App Router), TypeScript, Prisma, PostgreSQL, NextAuth, Tailwind, Vercel.

API routes	112
Screens	62
Rule modules in `src/lib`	45, of which 26 have unit tests
Database models / enums	44 / 15
Translation keys	1,227, in English and Arabic

The one structural idea

The rules are plain functions in `src/lib`, and everything else is a view onto them. Free-slot maths, how a visit was paid, what a rating averages to, when a birthday falls, who belongs in which customer group, how much of a deposit to ask for — each is a pure function with tests beside it, and the API routes are thin wrappers.

This is deliberate: a rule that lives inside a screen gets copied into the next screen and then the two diverge. It also means the awkward parts — a buffer around an appointment, a mixed payment that must add up, a clock that changes twice a year — are cheap to test and expensive to get wrong.

Decisions worth knowing

- **An appointment holds a chair; a visit is money.** They meet once, when the customer pays. An appointment can never become two visits.
- **Lines are snapshots.** A booking copies the service name, price and length as quoted. Raising a price never rewrites last week.
- **Time is shop-local.** Hours are stored as minutes from midnight against a weekday and resolved through `Africa/Cairo`, so they stay right when the clocks change. Everything stored is a UTC instant; only the boundaries convert.
- **Double booking is settled on save**, inside a serializable transaction, because free times are stale the moment they are shown.
- **Three doors** — staff, customer, barber — enforced in middleware *and* independently in every API route, because middleware protects pages, not data.
- **One set of design tokens.** Nothing in the customer or barber app names a colour of its own, which is what lets the whole thing turn from night to day by swapping one block of CSS.
- **A walk-in is a first-class citizen.** This shop books almost nothing and rings up over a thousand visits a month, so reviews, the usual, floor reports and the busyness forecast all read visits as well as appointments. Features that understood only appointments reached nobody here.

---

4. How it is worked on

Rules first. A new rule starts as a function in `src/lib` with a unit test, before any screen exists.

Every endpoint is tested for what it does *and* who it refuses. One of those refusal tests is what caught a customer being able to open the shop's own appointments screen.

Two suites, 950 tests.

Command	What runs	Takes
`npm test`	the rules, no database	seconds
`npm run test:all`	those plus the API against a real Postgres	~6 minutes

The integration suite runs against a real database in a container, not mocks — a mock cannot tell you that two people took the same slot, or that the drawer and the report disagree.

Every push runs CI: types, lint and the rules in one job; the whole API against Postgres in another; and a production build against an empty database, which proves the schema still applies to a fresh shop.

Three checks, and each has caught what the others missed. Tests pass code that `tsc` rejects. `tsc` passes code the production build rejects. CI runs in UTC and has caught timezone bugs invisible on a machine west of it.

Documentation is part of the change, not a follow-up. Every feature lands with the design note explaining why, and the roadmap is ticked only when something is built, tested *and* deployed.

---

5. Where it stands

Everything above is built, tested and deployed except the phone apps.

What remains is publishing the same app to Google Play and the App Store, which waits on the shop's own developer accounts ($25 once for Google, $99 a year for Apple). The accounts must be in the shop's name and only the owner can submit.

The honest part

Several finished features currently reach nobody, and it is worth saying plainly:

- **One customer has an app login**, out of 7,381. Loyalty, referrals, birthdays and reminders all work and all need an account. A QR on every printed receipt now points at sign-up; whether that moves the number is the open question.
- **No barber has a login yet**, out of 15. Their day screen, earnings, portfolio, time off and floor reports are finished and unreachable until they do.
- **The shop books nothing** — every one of those 1,210 monthly visits is a walk-in. The diary, reminders and deposits are built and idle. This is why so much of the recent work went into making features read visits rather than appointments.

None of that is a defect in the software. It is the difference between a system being finished and a system being adopted, and the second half is the harder one.

---

6. Seeing it

A demo, with invented data: https://escape-barbershop-demo.vercel.app

A separate deployment with its own database — sixteen invented customers, 526 visits, six weeks of plausible trade. Nothing real is in it. Logins are in the roadmap.

The live shop runs on a separate deployment behind its own login, and its database holds real customers. The two never meet.

The other documents

Document	Answers
Proposal	What we are building, for whom, and why it is one system rather than three
Design	How it is put together, in full: the data model, every rule, and the reason behind each decision
MVP	What "finished enough to use" means, and what is deliberately left out
Roadmap	Every phase, ticked or not, and what each needs from the shop
Testing	The two suites, what each file covers, and how to add to them

---